

Progetti Innovativi – Piattaforma Gioco HTML5

Questo progetto implementa un'infrastruttura completa per lo sviluppo e deployment di un gioco HTML5 singleplayer, con un'architettura di microservizi che include monitoring, logging e gestione utenti avanzata.



Sistema di Pulizia Aggiornato: Il progetto ora utilizza un [Sistema di Pulizia Centralizzato](#) intelligente con backup automatici e modalità multiple. I vecchi script `production-cleanup.sh` e `clean_build.sh` sono stati sostituiti da `cleanup.sh` più potente e sicuro.

Architettura del Sistema

Game Frontend

- **HTML5 Game Engine:** Frontend TypeScript/Vite per gioco singleplayer interattivo
- **Sandbox Environment:** Ambiente di sviluppo isolato per testing delle meccaniche
- **Asset Management:** Gestione dinamica di sprite, audio e risorse di gioco

Backend Game Services

- **Game API:** Server Go con gestione progressi giocatore, salvataggi e leaderboard
- **Session Management:** Gestione sessioni di gioco persistenti
- **Schema Validation:** Sistema di validazione richieste tramite JSON Schema

Game Data & Cache

- **MongoDB:** Database principale per profili giocatore, progressi e configurazioni
- **Redis:** Cache ad alta velocità per sessioni e stato temporaneo di gioco
- **Persistent Storage:** Salvataggio automatico progressi e achievement

Infrastructure & Monitoring

- **Elasticsearch:** Centralizzazione log di gioco e analytics comportamentali
- **Kibana:** Dashboard per monitoraggio performance e analytics giocatore
- **APM Server:** Monitoring performance lato server per ottimizzazione gameplay
- **Fluent Bit:** Aggregazione log da tutti i servizi di gioco
- **Elastic Agent:** Monitoring infrastruttura e resource usage

Networking & Proxy

- **NGINX Proxy Manager:** Load balancing e SSL per domini di gioco
- **Network Segmentation:** Isolamento traffico per sicurezza e performance
- **CDN Ready:** Configurazione ottimizzata per distribuzione globale



Requisiti

- Docker 20.10+
- Docker Compose 2.0+
- Node.js 18+ (per build locali)

- Almeno 4GB RAM per Elasticsearch e cache di gioco
-

Sviluppo

Per avviare l'ambiente di sviluppo completo:

```
./scripts/dev-reinstall.sh
```

Questo script:

- Reinstalla tutte le dipendenze
 - Builda frontend di gioco e sandbox di sviluppo
 - Configura database e cache per testing
 - Avvia tutti i servizi di monitoring
 - Configura proxy per accesso locale
-

Produzione

Pulizia Pre-Produzione

⚠ **Prima del deployment produzione**, esegui la pulizia selettiva dei dati di sviluppo:

```
./scripts/cleanup.sh --production
```

Questo comando:

- **Backup automatico** di tutte le configurazioni critiche
- **Rimuove solo dati di sviluppo**: Database giocatori, log, cache di testing
- **Preserva tutto il resto**: Domini, SSL, configurazioni servizi, dashboard Kibana
- **Verifica integrità** configurazioni dopo la pulizia
- **Output dettagliato** con conferme di sicurezza

Deployment Produzione

Per il deployment in produzione:

```
./scripts/prod-rebuild.sh
```

Questo script:

- Pulisce completamente l'ambiente di sviluppo
- Builda il frontend in modalità produzione ottimizzata
- Avvia i servizi usando `docker-compose.prod.yml` (senza sandbox)

- Configura monitoring completo per analytics
- Ottimizza performance per carico multi-utente

Workflow completo produzione:

1. `./scripts/cleanup.sh --production` - Pulizia selettiva dati sviluppo
2. `./scripts/check-persistent-config.sh` - Verifica configurazioni
3. `./scripts/prod-rebuild.sh` - Deploy ottimizzato

 **Nuovo Sistema:** Usa il [Sistema di Pulizia Centralizzato](#) per gestione avanzata e backup automatici

Nota: Lo script utilizza la configurazione produzione che esclude il sandbox di sviluppo e include ottimizzazioni specifiche per performance.

Aggiornamenti Rapidi

Aggiornamenti rapidi frontend in produzione:

```
./scripts/update_frontend.sh
```

Accesso ai Servizi

Applicazioni di Gioco

- **Game Frontend:** `http://localhost` (production game)
- **Game Development:** `http://localhost` (dev con hot reload)
- **Game API:** Endpoint REST accessibili dal frontend

Tools di Sviluppo

- **NGINX Proxy Manager:** `http://localhost:18081` (admin/changeme)
 - Gestione domini e SSL
 - Load balancing configuration

Monitoring & Analytics

- **Kibana:** `http://kibana.localhost` (elastic/m6OHmMuiqNrV1i25Jz3Z)
 - Game analytics dashboard
 - Performance monitoring
 - Player behavior analysis
- **APM Monitoring:** `http://apm.localhost`
 - Server performance metrics
 - API response times
 - Error tracking

Database Management

- **MongoDB Express:** <http://mongo-ui.localhost>
 - Player profiles management
 - Game data administration
 - **Redis Commander:** <http://redis-ui.localhost>
 - Session monitoring
 - Cache performance
-

Game Development Features

Singleplayer Experience

- Progressive gameplay con difficoltà adattiva
- Salvataggio automatico progressi locali
- Sistema achievement e unlock progressivi

Player Management

- Sessioni di gioco persistenti
- Profili giocatore con progression tracking
- Sistema salvataggio cloud-ready per continuità

Performance Optimization

- Cache intelligente per asset di gioco
- Preloading dinamico risorse per smooth gameplay
- Ottimizzazione rendering per 60fps costanti

Schema Validation System

- Validazione automatica API tramite JSON Schema ([comms/server/public/](#))
- Route whitelisting per sicurezza
- Structured error handling con codici specifici

Analytics & Monitoring

- Tracking eventi di gioco in tempo reale
- Analisi pattern comportamento giocatore
- Monitoring performance client-side
- Log aggregation per debugging e optimization

Logging & Analytics System

Il sistema di logging integrato permette monitoraggio completo del gioco:

Fluent Bit Integration:

- Aggregazione automatica log da tutti i servizi
- Identificazione container per filtering service-specific
- Parser specializzati per log MongoDB, NGINX, Go server

Elasticsearch Storage:

- Index `docker_logs` per log infrastruttura
- Structured logging con metadati enrichment
- Retention policies configurabili

Kibana Dashboards:

- **Game Performance:** Monitoring FPS, load times, crash rates
- **Player Analytics:** Progression tracking, completion rates
- **System Health:** Resource usage, API response times
- **Error Tracking:** Bug tracking e debugging assistance

Event Logging Examples:

```
// Frontend game events
analytics.logEvent('level_completed', {
  level: 3,
  completion_time: 45.2,
  score: 1250,
  player_deaths: 2
});

analytics.logEvent('achievement_unlocked', {
  achievement_id: 'first_boss_defeated',
  level: 5,
  session_duration: 1200
});
```

```
// Server structured logging
log.WithFields(logrus.Fields{
  "player_id": playerId,
  "action": "save_progress",
  "level": currentLevel,
  "score": newScore,
}).Info("Player progress saved successfully")
```

Configurazione

Environment Variables

Configurazione centralizzata in `.env`:

```
# Variabili principali per gaming
VITE_API_SERVER=http://server:3000    # Backend API
ELASTIC_APM_SERVICE_NAME=game-backend # Monitoring
```

```
MONGODB_HOST=db # Database giocatori
REDIS_URL=redis://app:password@redis # Cache sessioni

# NPM Registry per gestione pacchetti privati
NPM_EMAIL=dev.progettiinnovativi.2025@gmail.com
NPM_PASSWORD=MmkjJTaz@GTn8p
```

Game Configuration

- **APM:** Application Performance Monitoring configuration in [docker/volumes/README.md](#)
- **Logging:** Configurazione log di gioco in [docker/volumes/fluent-bit/](#)
- **Database:** Schema giocatori e punteggi via MongoDB Express
- **Proxy:** Domini personalizzati in NGINX Proxy Manager

Maintenance

Backup Game Data

```
# Backup automatico database giocatori
docker exec db mongodump --out /data/backup/$(date +%Y%m%d)
```

Cleanup Development

```
./scripts/clean-all.sh
```

Rimuove:

- Cache di sviluppo
- Container temporanei
- Build artifacts
- Log di sviluppo

Troubleshooting

Game Performance Issues

```
# Verifica stato servizi
docker-compose -f docker-compose.dev.yml ps

# Log specifici per debugging
docker logs server --tail 50 -f # API server logs
docker logs frontend --tail 50 -f # Frontend build logs
docker logs redis --tail 50 -f # Cache operations
```

Verifica Configurazione

```
./scripts/check-persistent-config.sh
```

Problemi Comuni Gaming

- **Lag multiplayer:** Controlla Redis performance e network latency
- **Asset loading slow:** Verifica cache frontend e CDN configuration
- **Player data loss:** Check MongoDB persistence e backup status
- **Authentication fails:** Verifica JWT token configuration e Redis sessions

Game Analytics

Player Metrics

- Session duration e retention
- Level completion rates
- Churn prediction

Technical Metrics

- API response times per endpoint
- Database query performance
- Cache hit rates
- Error tracking e debugging

Business Intelligence

- Revenue per player
- Feature usage analytics
- A/B testing results
- Performance benchmarking

Documentazione Tecnica

- [Game Architecture & Best Practices](#) - Guida completa sviluppo, pattern architetturali e best practices
- [Scripts Guide](#) - Automazione deployment e quick reference
- [Infrastructure Config](#) - Configurazioni servizi e volumi persistenti

Contributing

Development Workflow

1. **Setup ambiente:** `./scripts/dev-reinstall.sh`
2. **Implementa feature** in sandbox environment per prototipazione rapida
3. **Test su frontend reale:** Verifica funzionalità nel frontend principale, non solo sandbox
4. **Monitoring obbligatorio:** Configura APM tracking per nuove funzionalità
5. **Verifica analytics:** Controlla che eventi siano tracciati correttamente in Kibana
6. **Performance validation:** Esegui test performance e verifica metriche
7. **Deploy produzione:** `./scripts/prod-rebuild.sh` dopo validation completa

Code Quality Standards

- Tutti i commits devono passare performance tests completi
- **APM tracking obbligatorio** per ogni nuovo endpoint/feature
- **Analytics events** obbligatori per interazioni utente significative
- **Testing su frontend reale** richiesto, non solo in sandbox
- Documentation aggiornata per ogni modifica architetturale
- Schema validation aggiornata per nuovi endpoint API

Performance Requirements

- FPS minimo: 50fps (target: 60fps)
- Tempo caricamento livello: <2 secondi
- Risposta API: <100ms (95th percentile)
- Memory usage: <100MB per sessione di gioco

Riferimenti completi in [Game Architecture Guide](#).